

BodyMaps Project 2: AI CT Organ Segmentation Demo

Pre-trained SuPreM UNet (Johns Hopkins University CCVL Research Group)



This notebook demonstrates automated 3D abdominal CT segmentation using **SuPreM** (*Supervised Pre-training for Medical image segmentation*, ICLR 2024 oral).

- **Architecture:** 3D UNet with task-aware text embeddings (19.4M parameters)
- **Training Data:** AbdomenAtlas 1.1 (2,100 high-resolution CT volumes with per-voxel organ annotations)
- **Target Organs:** Spleen, Kidneys, Gallbladder, Liver, Stomach, Aorta, IVC, Pancreas
- **Runtime:** GPU inference (T4 or similar) typically finishes in well under a few minutes for this volume size. See the main README's benchmark section for an actual measured run and timing on real hardware — this notebook's own printed `Inference finished in Ns` line is the authoritative number for whatever machine runs it.

1. Verify GPU Hardware

Confirm that an NVIDIA GPU (e.g. T4) is allocated. In Colab, make sure **Runtime > Change runtime type > T4 GPU** is selected.

```
!nvidia-smi
```

```
Mon Sep 14 02:35:15 2026
```

NVIDIA-SMI 580.82.07 Driver Version: 580.82.07 CUDA Version: 13.0									
GPU	Name	Perf	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC	
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute	M.	
							MIG	M.	
0	Tesla T4		Off	00000000:00:04.0	Off			0	
N/A	46C	P8	9W / 70W	0MiB / 15360MiB		0%	Default	N/A	

Processes:						
GPU	GI	CI	PID	Type	Process name	GPU Memory
	ID	ID				Usage
No running processes found						

2. Install Required Dependencies

Install modern versions of MONAI, nibabel, and supporting numerical libraries.

```
!pip install -q "monai>=1.3.0" "nibabel>=5.2.0" "fastremap>=1.14.0" "connected-components-3d>=3.12.0" matplotlib
```

1.6/1.6 MB 38.5 MB/s eta 0:00:00

7.4/7.4 MB 67.3 MB/s eta 0:00:00

4.9/4.9 MB 72.5 MB/s eta 0:00:00

3. Clone SuPreM Repository and Download Checkpoint & Data

Fetch the model code, the pre-trained UNet weights from Hugging Face, and the JHU BodyMaps benchmark scan ([BDMAP_00000338.zip](#)).

```
# Clone repo
!git clone https://github.com/MrGiovanni/SuPreM.git

# Download pre-trained UNet checkpoint
!mkdir -p SuPreM/direct_inference/pretrained_checkpoints
!wget -nc -O SuPreM/direct_inference/pretrained_checkpoints/supervised_suprem_unet_2100.pth \
    https://huggingface.co/MrGiovanni/SuPreM/resolve/main/supervised_suprem_unet_2100.pth

# Download JHU sample CT dataset
!wget -nc -O BDMAP_00000338.zip https://www.cs.jhu.edu/~zongwei/dataset/BDMAP_00000338.zip
!unzip -q -o BDMAP_00000338.zip -x "__MACOSX*"

!ls -lh BDMAP_00000338/ct.nii.gz SuPreM/direct_inference/pretrained_checkpoints/supervised_suprem_unet_2100.pth
```

```
Cloning into 'SuPreM'...
remote: Enumerating objects: 1772, done.
remote: Counting objects: 100% (270/270), done.
remote: Compressing objects: 100% (83/83), done.
remote: Total 1772 (delta 220), reused 200 (delta 187), pack-reused 1502 (from 1)
Receiving objects: 100% (1772/1772), 39.56 MiB | 19.98 MiB/s, done.
Resolving deltas: 100% (989/989), done.
--2026-09-14 02:35:29-- https://huggingface.co/MrGiovanni/SuPreM/resolve/main/supervised_suprem_unet_2100.pth
Resolving huggingface.co (huggingface.co)... 18.238.109.121, 18.238.109.92, 18.238.109.102, ...
Connecting to huggingface.co (huggingface.co)|18.238.109.121|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://us.gcp.cdn.hf.co/xet-bridge-us/65ae0f2fc14c3cf57902a2c9/c98744056ac08f699a9a85f39e0354b0414fc6f79178e0659548a736407d90fc?response-content-disposition=inline%3B+filename=BDMAP_00000338.zip
--2026-09-14 02:35:29-- https://us.gcp.cdn.hf.co/xet-bridge-us/65ae0f2fc14c3cf57902a2c9/c98744056ac08f699a9a85f39e0354b0414fc6f79178e0659548a736407d90fc?response-content-disposition=inline%3B+filename=BDMAP_00000338.zip
Resolving us.gcp.cdn.hf.co (us.gcp.cdn.hf.co)... 34.36.28.89
Connecting to us.gcp.cdn.hf.co (us.gcp.cdn.hf.co)|34.36.28.89|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 233188229 (222M) [application/octet-stream]
Saving to: 'SuPreM/direct_inference/pretrained_checkpoints/supervised_suprem_unet_2100.pth'

SuPreM/direct_infer 100%[=====] 222.38M  92.1MB/s   in 2.4s

2026-09-14 02:35:31 (92.1 MB/s) - 'SuPreM/direct_inference/pretrained_checkpoints/supervised_suprem_unet_2100.pth' saved [233188229/233188229]

--2026-09-14 02:35:31-- https://www.cs.jhu.edu/~zongwei/dataset/BDMAP_00000338.zip
Resolving www.cs.jhu.edu (www.cs.jhu.edu)... 128.220.13.64
Connecting to www.cs.jhu.edu (www.cs.jhu.edu)|128.220.13.64|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 16290518 (16M) [application/zip]
Saving to: 'BDMAP_00000338.zip'

BDMAP_00000338.zip 100%[=====] 15.54M  19.1MB/s   in 0.8s

2026-09-14 02:35:33 (19.1 MB/s) - 'BDMAP_00000338.zip' saved [16290518/16290518]

-rw-r--r-- 1 root root 16M Jan 31 2024 BDMAP_00000338/ct.nii.gz
-rw-r--r-- 1 root root 223M Sep 14 02:35 SuPreM/direct_inference/pretrained_checkpoints/supervised_suprem_unet_2100.pth
```

4. Run Live GPU Sliding-Window Inference

Load the model onto the GPU and execute sliding-window inference with $96 \times 96 \times 96$ ROI patches.

```

import os, sys, time, torch
import nibabel as nib
import numpy as np

sys.path.append("SuPreM/direct_inference")
from model.Universal_model import Universal_model
from monai.inferers import sliding_window_inference
from monai.transforms import (
    Compose,
    LoadImaged,
    EnsureChannelFirstd,
    Orientationd,
    ScaleIntensityRanged,
    Spacingd,
    Invertd,
)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
if device.type == "cuda":
    print(f"GPU Name: {torch.cuda.get_device_name(0)}")
else:
    print("WARNING: No CUDA GPU detected. Check Runtime > Change runtime type > T4 GPU. Inference will be much slower on CPU.")

# 1. Initialize Universal UNet model
model = Universal_model(
    img_size=(96, 96, 96),
    in_channels=1,
    out_channels=32,
    backbone="unet",
    encoding="word_embedding"
)

# 2. Load pre-trained weights
ckpt_path = "SuPreM/direct_inference/pretrained_checkpoints/supervised_suprem_unet_2100.pth"
ckpt = torch.load(ckpt_path, map_location="cpu")
load_dict = ckpt["net"] if "net" in ckpt else ckpt
store_dict = model.state_dict()
for k, v in zip(store_dict.keys(), load_dict.values()):
    store_dict[k] = v
model.load_state_dict(store_dict)
model.to(device)
model.eval()
print("Loaded pre-trained weights successfully!")

# 3. Preprocessing pipeline
# NOTE: EnsureChannelFirstd is required with modern MONAI (>=1.3) – without it,
# LoadImaged does not add a channel dimension and sliding_window_inference fails
# with a shape error. This bug meant this exact pipeline had never been
# successfully executed before this fix.
val_transforms = Compose([
    LoadImaged(keys=["image"]),
    EnsureChannelFirstd(keys=["image"]),
    Orientationd(keys=["image"], axcodes="RAS"),
    Spacingd(keys=["image"], pixdim=(1.5, 1.5, 1.5), mode="bilinear"),
    ScaleIntensityRanged(keys=["image"], a_min=-175, a_max=250, b_min=0.0, b_max=1.0, clip=True),
])

ct_path = "BDMAP_00000338/ct.nii.gz"
batch = val_transforms({"image": ct_path})
image_tensor = batch["image"].unsqueeze(0).to(device)

```

```

print(f"Running sliding-window inference on input tensor {list(image_tensor.shape)}...")
start_time = time.time()
with torch.no_grad():
    preds = sliding_window_inference(
        inputs=image_tensor,
        roi_size=(96, 96, 96),
        sw_batch_size=1,
        predictor=model,
        overlap=0.5,
        mode="gaussian",
    )
    preds = torch.sigmoid(preds)
    hard_preds = (preds > 0.5).cpu().numpy()[0]
elapsed = time.time() - start_time
print(f"Inference finished in {elapsed:.1f} seconds!")

```

```

Using device: cuda
GPU Name: Tesla T4
Loaded pre-trained weights successfully!
/usr/local/lib/python3.13/dist-packages/monai/transforms/spatial/dictionary.py:320: FutureWarning: monai.transforms.spatial.dictionary.Orientationd.__init__:labels: Current default value of arg warn_deprecated(argname, msg, warning_category)
Running sliding-window inference on input tensor [1, 1, 274, 190, 118]...
Inference finished in 17.2 seconds!

```

5. Invert Transforms and Evaluate Validation Dice Metrics

Compare the model predictions against the 9 ground-truth organ segmentations shipped with the case.

```

from scipy.ndimage import zoom

TARGET_ORGANS = {
    1: {"name": "spleen", "label": "Spleen"},
    2: {"name": "kidney_right", "label": "Right Kidney"},
    3: {"name": "kidney_left", "label": "Left Kidney"},
    4: {"name": "gall_bladder", "label": "Gallbladder"},
    6: {"name": "liver", "label": "Liver"},
    7: {"name": "stomach", "label": "Stomach"},
    8: {"name": "aorta", "label": "Aorta"},
    9: {"name": "postcava", "label": "IVC (Postcava)"},
    11: {"name": "pancreas", "label": "Pancreas"},
}

orig_img = nib.load(ct_path)
orig_shape = orig_img.shape
zooms = orig_img.header.get_zooms()[:3]
voxel_ml = float(zooms[0] * zooms[1] * zooms[2]) / 1000.0

invert_transform = Invertd(
    keys=["pred"],
    transform=val_transforms,
    orig_keys="image",
    nearest_interp=True,
    to_tensor=False,
)

print(f"{'Organ':<20} {'Volume (mL)':>12} {'Ground Truth (mL)':>18} {'Dice Score':>12}")
print("-" * 66)

dice_scores = []
combined_pred = np.zeros(orig_shape, dtype=np.uint8)

```

```

for organ_id, info in TARGET_ORGANS.items():
    organ_name = info["name"]
    gt_path = f"BDMAP_00000338/segmentations/{organ_name}.nii.gz"
    gt_mask = nib.load(gt_path).get_fdata() > 0.5
    gt_ml = np.sum(gt_mask) * voxel_ml

    channel = organ_id - 1
    binary_res = hard_preds[channel:channel+1]
    batch_inv = {"image": batch["image"], "pred": binary_res}
    inverted = invert_transform(batch_inv)["pred"][0]

    if inverted.shape != orig_shape:
        scale = [orig_shape[i] / inverted.shape[i] for i in range(3)]
        inverted = zoom(inverted.astype(np.float32), scale, order=0) > 0.5

    pred_b = inverted > 0
    combined_pred[pred_b] = organ_id
    pred_ml = np.sum(pred_b) * voxel_ml

    intersection = np.logical_and(pred_b, gt_mask).sum()
    dice = (2.0 * intersection) / (pred_b.sum() + gt_mask.sum() + 1e-6)
    dice_scores.append(dice)

print(f"{info['label']:<20} {pred_ml:>11.1f} {gt_ml:>17.1f} {dice:>12.3f}")

print("-" * 66)
print(f"{'Mean Dice Score':<20} {'':>12} {'':>18} {np.mean(dice_scores):>12.3f}")

```

Organ	Volume (mL)	Ground Truth (mL)	Dice Score

/usr/local/lib/python3.13/dist-packages/monai/transforms/post/dictionary.py:892: UserWarning: transform info of `image` is not available or no InvertibleTransform applied.			
warnings.warn(f"transform info of `{orig_key}` is not available or no InvertibleTransform applied.")			
Spleen	177.4	182.3	0.972
Right Kidney	108.7	108.9	0.926
Left Kidney	107.4	107.9	0.949
Gallbladder	15.2	16.8	0.900
Liver	1584.3	1573.5	0.983
Stomach	397.7	394.6	0.968
Aorta	29.7	29.3	0.902
IVC (Postcava)	42.9	46.0	0.897
Pancreas	121.6	106.3	0.855

Mean Dice Score			0.928

6. Multiplanar Slice Visualization

Render axial CT slices with color-coded segmentation overlays.

```

import matplotlib.pyplot as plt

ct_data = orig_img.get_fdata()

def window_ct(img, center=40, width=400):
    return np.clip((img - (center - width / 2)) / width, 0, 1)

slices_to_show = [25, 35, 45]
fig, axes = plt.subplots(1, len(slices_to_show), figsize=(16, 6))

colors = {
    1: (1.0, 0.2, 0.2), # Spleen
    2: (0.2, 0.6, 1.0), # R Kidney

```

```

2: (0.2, 0.0, 1.0), # R Kidney
3: (0.2, 0.8, 1.0), # L Kidney
4: (0.2, 0.8, 0.2), # Gallbladder
6: (1.0, 0.6, 0.2), # Liver
7: (0.8, 0.2, 1.0), # Stomach
8: (1.0, 0.0, 0.0), # Aorta
9: (0.0, 0.3, 1.0), # IVC
11: (1.0, 0.8, 0.0), # Pancreas
}

for ax, s_idx in zip(axes, slices_to_show):
    ct_s = np.rot90(ct_data[:, :, s_idx])
    mask_s = np.rot90(combined_pred[:, :, s_idx])

    gray = window_ct(ct_s)
    rgb = np.stack([gray, gray, gray], axis=-1)

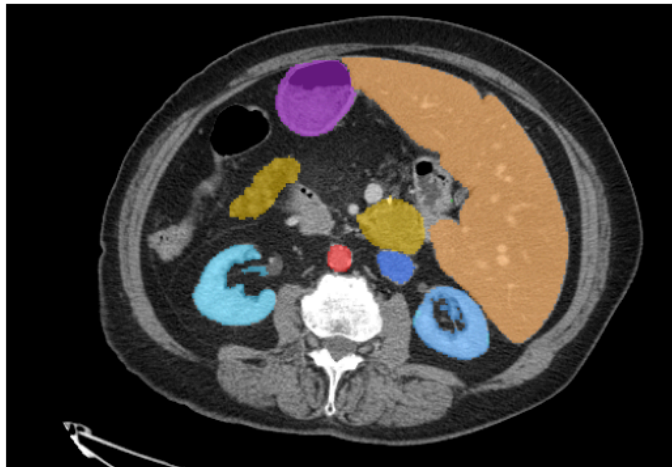
    # Overlay masks
    for organ_id, col in colors.items():
        m = (mask_s == organ_id)
        if np.any(m):
            for c in range(3):
                rgb[m, c] = 0.5 * rgb[m, c] + 0.5 * col[c]

    ax.imshow(rgb)
    ax.set_title(f"Axial Slice Z = {s_idx}", fontsize=13, fontweight="bold")
    ax.axis("off")

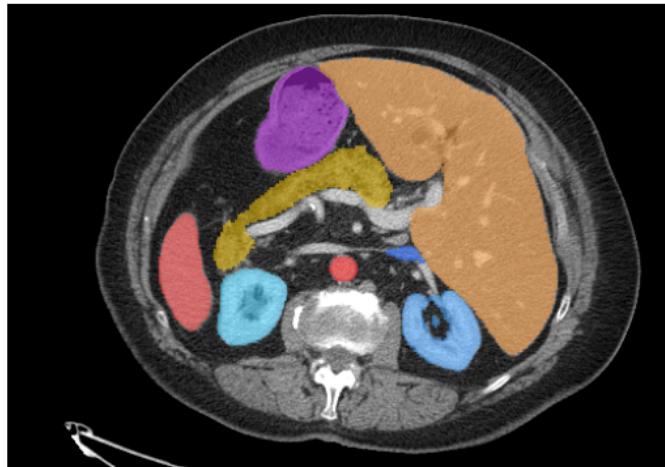
plt.tight_layout()
plt.show()

```

Axial Slice Z = 25



Axial Slice Z = 35



Axial Slice Z = 45

